

BEC305 Python Programming — Module 5 Quiz

Networked Programs, Web Services and Databases

Field	Value
Subject code	BEC305
Module	5 — Networked Programs, Web Services and Databases
Duration	45 minutes
Maximum marks	50 (Part A 20 + Part B 30)
Attempts allowed	1
Passing marks	20
Textbook	Al Sweigart, Automate the Boring Stuff with Python, 1st ed., 2015

Instructions

- Part A carries 20 one-mark multiple choice questions. Exactly one option is correct.
- Part B carries 13 short questions totalling 30 marks.
- Write SQL keywords in upper case and use ? placeholders for every value.
- State explicitly whether a value is bytes or str wherever it matters.
- No computers or interpreters are permitted during the quiz.

Course outcome and Bloom coverage

Band	Questions	Share	Target
L1–L2 Remember / Understand	7	21%	~30%
L3–L4 Apply / Analyse	19	58%	~45%
L5–L6 Evaluate / Create	7	21%	~25%

Course outcome spread: CO5: 33. Total questions: 33.

Part A — Multiple choice (20 × 1 mark = 20 marks)

A1. Which HTTP status code means the body is the resource you asked for? [C05 | L1 | 1 mark | easy | TB3 Ch. 12]

- (A) 200
- (B) 404
- (C) 500
- (D) 301

A2. What does it mean that HTTP is stateless? [C05 | L2 | 1 mark | medium | TB3 Ch. 12]

- (A) Each request carries everything the server needs; nothing is remembered between requests
- (B) The connection stays open indefinitely

- (C) The server stores a session for each client automatically
- (D) Requests must be sent in a fixed order

A3. Why must a socket receive loop continue until `recv` returns an empty byte string? [C05 | L4 | 1 mark | hard | TB3 Ch. 12]

- (A) TCP delivers a stream in arbitrary chunks, so one `recv` may not return the whole reply
- (B) `recv` always returns exactly one byte
- (C) The first `recv` returns only headers by definition
- (D) Python requires at least two `recv` calls

A4. What separates the headers from the body in an HTTP response? [C05 | L3 | 1 mark | medium | TB3 Ch. 12]

- (A) The first blank line
- (B) A semicolon
- (C) The Content-Length header value
- (D) A null byte

A5. Why must an image body never be decoded as text? [C05 | L4 | 1 mark | medium | TB3 Ch. 12]

- (A) Decoding corrupts the binary data and often raises `UnicodeDecodeError`
- (B) Images are always larger than the decode buffer
- (C) Python cannot decode more than 64 kB at once
- (D) Decoding is slower than writing directly

A6. What does `urllib.request.urlopen()` return? [C05 | L2 | 1 mark | medium | TB3 Ch. 12]

- (A) A file-like object yielding bytes
- (B) A string containing the page
- (C) A dictionary of headers
- (D) A socket

A7. What makes an XML document well-formed? [C05 | L2 | 1 mark | medium | TB3 Ch. 13]

- (A) Every tag closed, no overlapping nesting, exactly one root element
- (B) Every element carrying at least one attribute
- (C) All tags in lower case
- (D) A schema file being present

A8. What does `ElementTree.find()` return when the tag is absent? [C05 | L3 | 1 mark | medium | TB3 Ch. 13]

- (A) `None`
- (B) An empty `Element`
- (C) An empty string
- (D) It raises `KeyError`

A9. What does `findall()` return when no nodes match? [C05 | L3 | 1 mark | medium | TB3 Ch. 13]

- (A) An empty list
- (B) `None`
- (C) A list containing one `None`
- (D) It raises an exception

A10. A JSON object parses into which Python type? [C05 | L2 | 1 mark | easy | TB3 Ch. 13 up to 13.5]

- (A) dict
- (B) list
- (C) tuple
- (D) str

A11. What is a practical advantage of JSON over XML for this kind of work? [C05 | L4 | 1 mark | medium | TB3 Ch. 13 up to 13.5]

- (A) It parses directly into ordinary dictionaries and lists, so no tree API is needed
- (B) It supports attributes and comments
- (C) It enforces a schema automatically
- (D) It is a binary format

A12. Why is an indexed database table faster to search than a flat file? [C05 | L4 | 1 mark | medium | TB3 Ch. 15]

- (A) An index reaches a row in roughly logarithmic time; a file must be scanned end to end
- (B) Database files are smaller
- (C) Files cannot be searched at all
- (D) Databases keep everything in memory

A13. What distinguishes SQLite from a client-server database? [C05 | L2 | 1 mark | easy | TB3 Ch. 15]

- (A) The whole database is one file and the engine runs inside your process
- (B) It cannot store more than 1000 rows
- (C) It requires a separate server to be started
- (D) It does not support SQL

A14. What happens if you omit `conn.commit()` after inserting rows? [C05 | L3 | 1 mark | medium | TB3 Ch. 15]

- (A) The changes are discarded when the connection closes
- (B) The rows are saved anyway
- (C) An exception is raised at close
- (D) The database file is deleted

A15. Why should values be passed with `?` placeholders rather than string formatting? [C05 | L4 | 1 mark | hard | TB3 Ch. 15]

- (A) The database treats them as data rather than SQL, preventing injection and handling quoting
- (B) Placeholders are faster to type
- (C) String formatting is a `SyntaxError` in SQL
- (D) Placeholders compress the query

A16. What does an `UPDATE` statement without a `WHERE` clause do? [C05 | L3 | 1 mark | medium | TB3 Ch. 15]

- (A) Updates every row in the table
- (B) Updates nothing
- (C) Raises an error
- (D) Updates only the first row

A17. What is a primary key? [C05 | L2 | 1 mark | medium | TB3 Ch. 15]

- (A) An integer the database assigns, meaningless outside the table and never reused
- (B) The column a user searches by
- (C) Another table's identifier stored in this table
- (D) The first column of the table

A18. Why should a foreign key hold the primary key rather than the logical key? [C05 | L5 | 1 mark | hard | TB3 Ch. 15]

- (A) Renaming the logical value then requires changing one row rather than every referring row
- (B) Integers sort faster
- (C) Logical keys cannot be indexed
- (D) Foreign keys must be integers by rule

A19. What does a JOIN without an ON clause produce? [C05 | L5 | 1 mark | hard | TB3 Ch. 15]

- (A) Every combination of rows from both tables
- (B) An empty result
- (C) An error
- (D) Only matching rows, by guessing the columns

A20. What makes a spider resumable after an interruption? [C05 | L5 | 1 mark | hard | TB3 Ch. 15]

- (A) A retrieved flag stored in the table and committed after each record
- (B) A larger recv buffer
- (C) Running the fetch loop faster
- (D) Keeping the visited set in a Python variable

Part B — Descriptive (13 questions, 30 marks)

B1. Describe the structure of an HTTP request and response, and explain why the status code must be checked first. [C05 | L3 | 3 marks | TB3 Ch. 12]

B2. Explain why a socket receive loop is needed, and what signals the end of the reply. [C05 | L4 | 3 marks | TB3 Ch. 12]

B3. Explain the three kinds of key and the normalisation rule that follows from them. [C05 | L5 | 3 marks | TB3 Ch. 15]

B4. Explain what a JOIN does, what the ON clause is for, and what happens without it. [C05 | L5 | 3 marks | TB3 Ch. 15]

B5. Why must an image body be kept as bytes, and how are the headers separated from it? [C05 | L4 | 2 marks | TB3 Ch. 12]

B6. What does urlopen give you over a raw socket, and what does it not hide? [C05 | L3 | 2 marks | TB3 Ch. 12]

B7. Explain the hazard in chaining .text onto ElementTree find(). [C05 | L4 | 2 marks | TB3 Ch. 13]

B8. Contrast find() with findall() and say which is the safer default for a collection. [C05 | L3 | 2 marks | TB3 Ch. 13]

B9. Describe how JSON maps onto Python types and what that means for the code you write. [C05 | L3 | 2 marks | TB3 Ch. 13 up to 13.5]

B10. Why is a database preferable to a flat file once the data grows? [C05 | L4 | 2 marks | TB3 Ch. 15]

B11. Write a parameterised INSERT and explain why string formatting must not be used instead. [C05 | L4 | 2 marks | TB3 Ch. 15]

B12. Explain what makes a spider resumable and idempotent. [C05 | L5 | 2 marks | TB3 Ch. 15]

B13. Describe the four stages of the end-to-end pipeline and why keeping them separate matters. [C05 | L5 | 2 marks | TB3 Ch. 12-15]

Answer key — Part A

Q	Answer	CO	Bloom	Reference
A1	A	CO5	L1	TB3 Ch. 12
A2	A	CO5	L2	TB3 Ch. 12
A3	A	CO5	L4	TB3 Ch. 12
A4	A	CO5	L3	TB3 Ch. 12
A5	A	CO5	L4	TB3 Ch. 12
A6	A	CO5	L2	TB3 Ch. 12
A7	A	CO5	L2	TB3 Ch. 13
A8	A	CO5	L3	TB3 Ch. 13
A9	A	CO5	L3	TB3 Ch. 13
A10	A	CO5	L2	TB3 Ch. 13 up to 13.5
A11	A	CO5	L4	TB3 Ch. 13 up to 13.5
A12	A	CO5	L4	TB3 Ch. 15
A13	A	CO5	L2	TB3 Ch. 15
A14	A	CO5	L3	TB3 Ch. 15
A15	A	CO5	L4	TB3 Ch. 15
A16	A	CO5	L3	TB3 Ch. 15
A17	A	CO5	L2	TB3 Ch. 15
A18	A	CO5	L5	TB3 Ch. 15
A19	A	CO5	L5	TB3 Ch. 15
A20	A	CO5	L5	TB3 Ch. 15

Part A — why each option is right or wrong

A1. Correct answer: (A) 200

- (A) Correct - anything else means the body is something other than the requested resource.

- (B) Wrong - 404 means not found, and the body is an error page.
- (C) Wrong - 500 is a server error.
- (D) Wrong - 301 is a permanent redirect, so the resource is elsewhere.

A2. Correct answer: (A) Each request carries everything the server needs; nothing is remembered between requests

- (A) Correct - cookies and tokens exist to reintroduce the state the protocol omits.
- (B) Wrong - statelessness concerns memory between requests, not connection lifetime.
- (C) Wrong - that is exactly what the protocol does not do by itself.
- (D) Wrong - requests are independent, so no ordering is imposed.

A3. Correct answer: (A) TCP delivers a stream in arbitrary chunks, so one recv may not return the whole reply

- (A) Correct - the empty return is the signal that the server has closed its end.
- (B) Wrong - recv returns up to the buffer size, which is usually many bytes.
- (C) Wrong - the split between headers and body does not align with chunk boundaries.
- (D) Wrong - the number of calls depends entirely on the response size and the network.

A4. Correct answer: (A) The first blank line

- (A) Correct - HTTP defines exactly one empty line between them.
- (B) Wrong - semicolons appear inside some header values but separate nothing structurally.
- (C) Wrong - Content-Length tells you how long the body is, not where it starts.
- (D) Wrong - HTTP headers are text and contain no null bytes.

A5. Correct answer: (A) Decoding corrupts the binary data and often raises UnicodeDecodeError

- (A) Correct - binary bytes are not characters, so they must stay bytes all the way to the file.
- (B) Wrong - size is irrelevant; the problem is the interpretation.
- (C) Wrong - there is no such limit.
- (D) Wrong - speed is not the issue; correctness is.

A6. Correct answer: (A) A file-like object yielding bytes

- (A) Correct - it is read or iterated like a file, but each line must be decoded.
- (B) Wrong - the content is bytes and must be decoded explicitly.
- (C) Wrong - headers are available from the object but are not what it returns.
- (D) Wrong - the socket is what urlopen hides from you.

A7. Correct answer: (A) Every tag closed, no overlapping nesting, exactly one root element

- (A) Correct - a parser rejects anything else rather than guessing.
- (B) Wrong - attributes are optional.
- (C) Wrong - XML is case sensitive but imposes no case convention.
- (D) Wrong - a schema governs validity, which is a stricter and separate notion.

A8. Correct answer: (A) None

- (A) Correct - which is why chaining .text onto it raises AttributeError on documents that differ from your sample.
- (B) Wrong - no Element is created for a tag that is not there.
- (C) Wrong - find returns an Element or None, never a string.
- (D) Wrong - find signals absence by its return value, not by raising.

A9. Correct answer: (A) An empty list

- (A) Correct - so a loop over it simply runs zero times, which makes findall the safer default for collections.
- (B) Wrong - None is what find returns; findall always returns a list.
- (C) Wrong - an empty list has no elements at all.
- (D) Wrong - an empty result is a normal outcome, not an error.

A10. Correct answer: (A) dict

- (A) Correct - and a JSON array parses into a list, which is why no tree API is needed.
- (B) Wrong - that is what a JSON array becomes.
- (C) Wrong - json never produces tuples.
- (D) Wrong - the text is what you started with; loads produces structure.

A11. Correct answer: (A) It parses directly into ordinary dictionaries and lists, so no tree API is needed

- (A) Correct - the parsed result is reached with plain subscripting.
- (B) Wrong - JSON has neither attributes nor comments; that is a point in XML favour.
- (C) Wrong - JSON enforces nothing by itself.
- (D) Wrong - JSON is text, like XML.

A12. Correct answer: (A) An index reaches a row in roughly logarithmic time; a file must be scanned end to end

- (A) Correct - the gap widens with every record added.
- (B) Wrong - a database file is usually larger, because the index costs space.
- (C) Wrong - files can be searched; the cost is what differs.
- (D) Wrong - a database reads from disk like anything else.

A13. Correct answer: (A) The whole database is one file and the engine runs inside your process

- (A) Correct - which is why there is nothing to install or configure.
- (B) Wrong - there is no such limit.
- (C) Wrong - that is the client-server model SQLite avoids.
- (D) Wrong - SQLite supports most of standard SQL.

A14. Correct answer: (A) The changes are discarded when the connection closes

- (A) Correct - statements run inside a transaction that must be committed to persist.
- (B) Wrong - the whole point of a transaction is that uncommitted work is provisional.
- (C) Wrong - closing without committing is silent, which is what makes it a trap.
- (D) Wrong - the file remains; it simply does not contain the new rows.

A15. Correct answer: (A) The database treats them as data rather than SQL, preventing injection and handling quoting

- (A) Correct - a value containing a quote otherwise breaks out of the string and becomes executable SQL.
- (B) Wrong - convenience is not the point; safety is.
- (C) Wrong - string-built SQL is valid Python and valid SQL, which is why the vulnerability exists.
- (D) Wrong - query size is irrelevant.

A16. Correct answer: (A) Updates every row in the table

- (A) Correct - and there is no undo, which is why the WHERE is never optional in practice.

- (B) Wrong - the absence of a restriction means no rows are excluded.
- (C) Wrong - it is entirely valid SQL.
- (D) Wrong - SQL has no implicit row limit.

A17. Correct answer: (A) An integer the database assigns, meaningless outside the table and never reused

- (A) Correct - its meaninglessness is what makes it a stable target for references.
- (B) Wrong - that is the logical key.
- (C) Wrong - that is the foreign key.
- (D) Wrong - column position has nothing to do with it.

A18. Correct answer: (A) Renaming the logical value then requires changing one row rather than every referring row

- (A) Correct - and any referring row missed during a rename silently becomes a separate entity.
- (B) Wrong - sort speed is a minor consideration next to update integrity.
- (C) Wrong - logical keys are routinely indexed and should carry UNIQUE.
- (D) Wrong - the database permits other types; the reason is maintenance, not rule.

A19. Correct answer: (A) Every combination of rows from both tables

- (A) Correct - for two 1000-row tables that is a million rows; the row count is the quickest diagnostic.
- (B) Wrong - it returns far more rows, not fewer.
- (C) Wrong - a cross join is valid SQL and runs happily.
- (D) Wrong - SQL never guesses the join condition.

A20. Correct answer: (A) A retrieved flag stored in the table and committed after each record

- (A) Correct - the flag survives the process ending, so the next run selects where the last one stopped.
- (B) Wrong - buffer size affects one read, not resumability.
- (C) Wrong - speed does not help once the process has ended.
- (D) Wrong - an in-memory set dies with the process, which is exactly the problem.

Part B — model answers and marking scheme

B1 (3 marks) — CO5, L3, TB3 Ch. 12

Question. Describe the structure of an HTTP request and response, and explain why the status code must be checked first.

Model answer. The client opens a TCP connection and sends a request line naming a method and a path, followed by headers and a blank line. The server replies with a status line, headers, a blank line and then the body. The status code must be read before the body is used, because a 404 reply still has a body — an error page — and code that skips the check happily parses that page as though it were the requested data. HTTP is also stateless, so each request carries everything the server needs and nothing is remembered between them.

Point required	Marks
Correct structure of request and response	1
Correct explanation of why the status code is checked first	1
Correct statement of statelessness	1

Point required	Marks
Total	3

B2 (3 marks) — CO5, L4, TB3 Ch. 12

Question. Explain why a socket receive loop is needed, and what signals the end of the reply.

Model answer. TCP delivers a byte stream, not messages, so a single `recv` call returns whatever has arrived so far, up to the buffer size. The reply therefore arrives in chunks of arbitrary size and must be accumulated in a loop. The loop ends when `recv` returns an empty byte string, which is the signal that the server has closed its end of the connection. Code that calls `recv` once typically captures the status line, the headers and only a fragment of the body.

Point required	Marks
Correct explanation of the stream and arbitrary chunking	1
Correct termination condition	1
Correct description of what a single <code>recv</code> would capture	1
Total	3

B3 (3 marks) — CO5, L5, TB3 Ch. 15

Question. Explain the three kinds of key and the normalisation rule that follows from them.

Model answer. A primary key is an integer the database assigns; it is meaningless outside the table and never reused, which makes it a stable target for references. A logical key is the value the outside world uses to identify a row, such as an email address, and should carry a `UNIQUE` constraint. A foreign key is a column holding another table's primary key and is how one row points at another. The rule that follows is never to repeat a string value across rows: store it once in its own table and reference it by integer primary key, so a correction touches one cell rather than many.

Point required	Marks
All three keys defined correctly	2
Correct normalisation rule with its maintenance justification	1
Total	3

B4 (3 marks) — CO5, L5, TB3 Ch. 15

Question. Explain what a JOIN does, what the ON clause is for, and what happens without it.

Model answer. A JOIN reassembles at query time the rows that normalisation split across tables. The ON clause states the matching condition, so `ON Track.artist_id = Artist.id` pairs each track with its one artist and returns one row per track. Without an ON clause the query produces a cross join, pairing every row of one table with every row of the other, so three tracks and two artists return six rows rather than three. For two thousand-row tables that is a million rows, and the row count of the result is the quickest way to detect the missing condition.

Point required	Marks
Correct purpose of JOIN and the ON clause	1

Point required	Marks
Correct description of the cross-join outcome with numbers	1
Row count named as the diagnostic	1
Total	3

B5 (2 marks) — CO5, L4, TB3 Ch. 12

Question. Why must an image body be kept as bytes, and how are the headers separated from it?

Model answer. An image body is binary data, not characters, so decoding it as text corrupts it and frequently raises `UnicodeDecodeError` partway through. The headers and body arrive in one stream and are separated by the first blank line, which HTTP guarantees appears exactly once between them. The header portion alone is decoded to read `Content-Type` and `Content-Length`; everything after the blank line stays as bytes and is written to a file opened in binary mode, since text mode would translate line endings and destroy the file.

Point required	Marks
Correct reason for keeping the body as bytes	1
Correct separation rule and binary file mode	1
Total	2

B6 (2 marks) — CO5, L3, TB3 Ch. 12

Question. What does `urlopen` give you over a raw socket, and what does it not hide?

Model answer. `urlopen` performs the connection, the request, the receive loop and the header split, and returns an object that can be read or iterated like a file, so the protocol stops being your concern. It also raises `HTTPError` on an error status rather than handing back the error page, which is the behaviour you want. What it does not hide is the bytes-versus-text distinction: lines arrive as bytes and must be decoded, because `urlopen` does not know the encoding of an arbitrary resource.

Point required	Marks
Correct account of what is abstracted away	1
Correct statement that decoding remains the callers responsibility	1
Total	2

B7 (2 marks) — CO5, L4, TB3 Ch. 13

Question. Explain the hazard in chaining `.text` onto `ElementTree find()`.

Model answer. `find()` returns an `Element` when the tag is present and `None` when it is absent, so `tree.find('email').text` raises `AttributeError: 'NoneType' object has no attribute 'text'` on any document lacking that optional element. The pattern works on the sample document used during development and fails on the first real record that omits the tag, which means the failure arrives in production. The remedy is to bind the result, test it against `None`, and only then read `.text`.

Point required	Marks
Correct explanation of the <code>None</code> return and resulting exception	1

Point required	Marks
Correct remedy and the reason the bug survives testing	1
Total	2

B8 (2 marks) — CO5, L3, TB3 Ch. 13

Question. Contrast `find()` with `findall()` and say which is the safer default for a collection.

Model answer. `find()` returns the first matching Element or None, so it suits a single expected value. `findall()` returns a list of every match, which may be empty, so it suits a repeating structure. Because an absent tag yields an empty list rather than None, a loop over `findall` simply runs zero times where a chained `find` would have raised `AttributeError`. `findall` is therefore the safer default whenever the data is a collection, and `len()` on its result counts the records without a second parse.

Point required	Marks
Correct return values of both	1
Correct safety argument for collections	1
Total	2

B9 (2 marks) — CO5, L3, TB3 Ch. 13 up to 13.5

Question. Describe how JSON maps onto Python types and what that means for the code you write.

Model answer. `json.loads` turns a JSON object into a dictionary, an array into a list, strings into `str`, numbers into `int` or `float`, `true` and `false` into `True` and `False`, and `null` into `None`. The parsed result is therefore ordinary Python data, reached with the same subscripting and iteration used for any dictionary or list, so no tree API is required. `json.dumps` performs the reverse. Real feeds are irregular, so keys that are not guaranteed by the schema should be read with `get()` and a default rather than square brackets.

Point required	Marks
Correct type mapping	1
Correct consequence for the code, including the <code>get()</code> point	1
Total	2

B10 (2 marks) — CO5, L4, TB3 Ch. 15

Question. Why is a database preferable to a flat file once the data grows?

Model answer. A flat file must be scanned end to end to find a record, which averages $n/2$ comparisons and costs n for a miss; among 100000 records that is around 50000 comparisons. An indexed table descends a balanced structure instead and reaches the row in roughly 17 steps, with a miss costing the same. The database also enforces that every row of a table has the same columns and persists without any serialisation code. The price is a schema that must be designed before data can be stored.

Point required	Marks
Correct complexity contrast with numbers	1
Correct statement of structure enforcement and the schema cost	1
Total	2

B11 (2 marks) — CO5, L4, TB3 Ch. 15

Question. Write a parameterised INSERT and explain why string formatting must not be used instead.

Model answer. Write `cur.execute('INSERT INTO Users (email, name) VALUES (?, ?)', (email, name))`, passing the values as a tuple. The database then treats them as data rather than as SQL, which handles quoting correctly and makes injection impossible. Building the statement by concatenation means a value containing a quote closes the string early and the remainder is executed as SQL, so a crafted name could drop a table. The formatted version is valid Python and valid SQL, which is precisely why the vulnerability survives review.

Point required	Marks
Correct parameterised statement with the tuple	1
Correct injection explanation	1
Total	2

B12 (2 marks) — CO5, L5, TB3 Ch. 15

Question. Explain what makes a spider resumable and idempotent.

Model answer. Resumability comes from storing a retrieved flag in the table and committing after each record, so a run interrupted after two hours restarts by selecting the first row where retrieved is 0 rather than fetching everything again. Idempotence comes from a UNIQUE constraint on the logical key together with INSERT OR IGNORE, so offering an already-stored record adds nothing and the row count after a second run is unchanged. Committing once at the end would lose the entire crawl on an interruption, which is why the per-record commit is worth its small cost.

Point required	Marks
Correct resumability mechanism	1
Correct idempotence mechanism	1
Total	2

B13 (2 marks) — CO5, L5, TB3 Ch. 12-15

Question. Describe the four stages of the end-to-end pipeline and why keeping them separate matters.

Model answer. The stages are fetch over HTTP, parse the XML or JSON into Python structures, store into normalised tables, and query with SQL. Keeping them separate makes each testable with a fixed input: a saved response file lets the parser be exercised with no network at all, and a fixture structure lets the storage layer be tested without either. It also means the expensive stage, fetching, is performed once while the cheap stage, querying, can be repeated freely. Interleaving fetch and parse blocks all development during a service outage.

Point required	Marks
All four stages named in order	1
Correct testability and cost argument for separation	1
Total	2